Software Requirements Specification (SRS)

Project: RepoRadar — Automated Open-Source Issue Matcher & Engagement Platform

Document Version: 1.0.0 (Production Release) **Author:** Lead Systems Architect
Target Stack: Next.js 15, TypeScript, PostgreSQL, Node.js Workers

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive architectural and functional definition for **RepoRadar**, a developer tool designed to automate the discovery, curation, and algorithmic scoring of beginner-friendly open-source issues across GitHub. This document serves as the authoritative blueprint for Phase 2 (High-Level Design) through Phase 5 (Production Deployment) of the Software Development Life Cycle (SDLC).

1.2 Project Scope

RepoRadar bridges the gap between aspiring open-source contributors (particularly computer science undergraduates and junior developers) and active open-source maintainers. The system autonomously ingests issues from public repositories, evaluates their solvability using a proprietary scoring engine, stores them in a high-speed indexed relational database, and delivers tailored recommendations via both an interactive web dashboard and automated email digests.

1.3 Definitions, Acronyms, and Abbreviations

TERM / ACRONYM	DEFINITION
SDLC	Software Development Life Cycle — A structured engineering framework for developing software with high quality and predictable timelines.
ETag / Conditional Read	HTTP response headers used by GitHub's API to validate cached responses without consuming hourly rate limits if data hasn't changed (`304 Not Modified`).
Solvability Score	A computed numerical metric (0.0 to 100.0) predicting the likelihood that an external beginner can successfully resolve a specific open issue without getting blocked.
RBAC	Role-Based Access Control — Restricting network or system privileges based on user roles (e.g., Contributor vs. Admin).
Idempotency	An architectural property ensuring that executing a background task multiple times yields identical database state without creating duplicate records.

2. OVERALL DESCRIPTION

2.1 Product Perspective & Architectural Style

RepoRadar is architected as a **Modular Monolith** utilizing the Next.js App Router ecosystem. To guarantee strict **Separation of Concerns (SoC)**, the system is divided into four distinct operational layers:

- **Presentation Layer:** React Server Components (RSC) and Client Components styled with Tailwind CSS for high-speed rendering and SEO optimization.
- **Service & API Layer:** Type-safe REST endpoints and Server Actions enforcing business validation, authentication, and user matching logic.
- **Data Access Layer (DAL):** An abstracted Repository Pattern implemented via Prisma/Drizzle ORM communicating with a cloud-hosted PostgreSQL database (Neon / Supabase).
- **Background Worker & Ingestion Layer:** Decoupled Node.js cron workers executing scheduled ingestion pipelines, external GitHub API polling, and email dispatching without blocking web server threads.

2.2 Operating Environment & Constraints

> **CRITICAL SYSTEM CONSTRAINT: GITHUB API RATE LIMITING**
>
> GitHub enforces strict rate limits: 60 requests/hour for unauthenticated IPs and 5,000 requests/hour for authenticated tokens. The ingestion pipeline **must never** execute raw search queries per user request. All reads must occur against our internal PostgreSQL B-Tree/GIN indexed tables. External API polling must utilize OAuth tokens, rotating keys, and ETag conditional headers.

3. SYSTEM ARCHITECTURE & DESIGN PRINCIPLES

To ensure enterprise-grade maintainability, RepoRadar strictly adheres to the following foundational engineering principles:

- **SOLID Principles:** Services are built around Single Responsibility (e.g., `ScoringService` is isolated from `IngestionService`) and Dependency Inversion (UI depends on interface abstractions, not raw database clients).
- **Repository Pattern:** Direct database queries are prohibited inside UI components or route handlers. All database mutations pass through domain-specific repository classes (e.g., `IssueRepository.findMatchingIssues(userPrefs)`).
- **Fault Tolerance & Exponential Backoff:** If an external API (GitHub or Email Provider) throttles requests or fails, background workers apply exponential backoff with jitter to prevent cascading system failures.

4. DETAILED FUNCTIONAL REQUIREMENTS (FRS)

4.1 Automated Ingestion & Synchronization Engine

`FR-1.1` The system shall execute background cron jobs at scheduled intervals (e.g., every 4 hours) to query the GitHub REST/GraphQL APIs for issues containing target labels: `good first issue`, `help wanted`, `beginner`, and `documentation`.

`FR-1.2` The ingestion engine shall implement **ETag caching**. For previously scraped repositories, the worker must store the last received ETag header and transmit it via `If-None-Match` on subsequent polls, updating internal records only upon receiving an HTTP `200 OK`.

`FR-1.3` The ingestion engine shall be strictly **idempotent**. Upsert operations (Insert or Update) must utilize the unique composite constraint on `(repository_id, github_issue_number)` to prevent duplication.

4.2 Solvability & Stale Scoring Engine

`FR-2.1` Every ingested issue shall be processed by an algorithmic evaluation engine that computes a **Solvability Score** (S) using the following weighted mathematical model:

$$S = w_1 \cdot (100 - Age_{days}) + w_2 \cdot (RepoHealth) - w_3 \cdot (ActiveComments) - w_4 \cdot (OpenPRs)$$

where:
• **Age_{days}**: Elapsed days since issue creation (decay factor; older issues are more likely to be abandoned or complex).
• **$RepoHealth$**: A normalized score (0–100) derived from maintainer responsiveness, merge velocity, and recent commit activity over the last 30 days.
• **$ActiveComments$**: Penalty factor representing competition; issues with >5 comments often indicate existing developers are already working on a solution.
• **$OpenPRs$**: Penalty applied if a Pull Request referencing the issue number already exists in the repository.

4.3 High-Speed Query & Indexing Core

`FR-3.1` Users shall be able to filter issues by programming language, frameworks (e.g., React, Django, Docker), solvability score threshold, and repository star count with sub-100ms response times.

`FR-3.2` To support real-time searching without external API latency, the database schema shall implement a **GIN (Generalized Inverted Index)** on the array of issue labels/tech tags and a **Composite B-Tree Index** on `(language, solvability_score DESC, is_open)`.

4.4 User Profiles & Bookmarking Workflow

`FR-4.1` The platform shall authenticate users via GitHub OAuth 2.0, capturing their public username, avatar, and primary repository language statistics.

`FR-4.2` Authenticated users shall be able to define custom profile preferences, including preferred tech stacks, target difficulty levels, and exclusion rules (e.g., ignore repositories with < 100 stars).

`FR-4.3` Users shall be able to bookmark issues into personal workspaces with status tags: `Saved`, `Investigating`, `PR Submitted`, and `Resolved`.

4.5 Multi-Channel Personalized Digest System (Web & Email)

> **ARCHITECTURAL FOCUS: DUAL-DELIVERY NOTIFICATION ENGINE**
>
> To maximize user engagement, the personalized recommendation engine must deliver insights seamlessly across two synchronized mediums: an interactive real-time web dashboard feed and automated HTML email summaries.

`FR-5.1 (Web Dashboard Feed)` The web application shall provide a dedicated "Daily Recommendations" interface. Upon loading, the system queries PostgreSQL for the top 10 highest-scoring issues that match the authenticated user's exact profile tags, excluding issues they have already bookmarked or dismissed.

`FR-5.2 (Automated Email Delivery Engine)` A dedicated background scheduler shall run daily at 08:00 UTC, executing the following sequential workflow:

1. Query the `Users` table for active users where `email_notifications_enabled = true` and `digest_frequency = 'DAILY'`.
2. For each eligible user, execute a specialized SQL matching query joining `Issues` against `UserPreferences` to extract the top 5 unresolved, highest-solvability issues discovered within the last 24 hours.
3. Compile the matched data into a responsive, mobile-optimized HTML email template featuring issue titles, direct GitHub repository links, solvability scores, and required tech stack badges.
4. Dispatch the payload through an SMTP/API provider (e.g., Resend, AWS SES, or SendGrid) using asynchronous batching to prevent memory exhaustion.

`FR-5.3 (Digest Idempotency & Logging)` To prevent duplicate email dispatches, the notification worker shall log every sent email in a `DigestLogs` transaction table containing `(user_id, issue_id, sent_timestamp)`. The matching engine must explicitly filter out any `issue_id` present in the user's digest log within the preceding 14 days.

5. NON-FUNCTIONAL REQUIREMENTS (NFRS)

CATEGORY	MEASURABLE ENGINEERING STANDARD
NFR-1: Read Performance	95% of database queries for dashboard feeds and search filters must complete in less than 50ms. Page load Time to Interactive (TTI) must be under 1.2 seconds on standard broadband connections.
NFR-2: Ingestion Resilience	Background ingestion jobs must gracefully handle network timeouts, HTTP 5xx errors from GitHub, and rate-limit headers (`X-RateLimit-Remaining`) by pausing execution until the timestamp indicated by `X-RateLimit-Reset` .
NFR-3: Data Storage & Archival	To maintain optimal database index size, issues marked as closed or inactive for > 60 days shall be automatically moved to an analytical cold-storage archive table (`issues_archive`).
NFR-4: Security & Privacy	All user OAuth tokens and session cookies must be encrypted at rest using AES-256-GCM and transmitted strictly over TLS 1.3 (HTTPS). Email unsubscribe links must generate cryptographic JWT tokens valid for 30 days without requiring sign-in.

6. HIGH-LEVEL DATA ENTITIES (PHASE 2 PREVIEW)

The system relational schema revolves around six primary tables designed for relational integrity and high-performance querying:

- `users` : Core identity, OAuth metadata, and global notification flags.
- `user_preferences` : 1-to-1 extension table storing tech stack arrays, star thresholds, and digest schedules.
- `repositories` : GitHub repo metadata (stars, forks, open issues count, health score, ETag hash).
- `issues` : Central repository of scraped issues (title, url, body markdown, solvability score, tech tags array, state).
- `user_bookmarks` : Join table tracking user interactive states (saved, PR open, resolved) per issue.
- `digest_logs` : Audit trail enforcing idempotency for email and webhook notification deliveries.